

Clo-Author: rutina paso a paso

De la idea al paper con agentes de IA en Claude Code

Ana Laura Catelén · Facultad de Ciencias Económicas y Sociales, UNMDP

Guía basada en la ponencia «Automatizar sin perder el criterio: experiencias con IA en el proceso de investigación económica», presentada en el CIGU 2026

Uso de inteligencia artificial

Los pasos de esta rutina se armaron con la asistencia de inteligencia artificial generativa (Claude, de Anthropic), a partir de la ponencia de la autora y de las fuentes citadas. La autora revisó el contenido y es responsable de lo que dice.

1. Mapa general de la rutina

Diez pasos, de la instalación al envío. Los comandos se corren en este orden, y cada etapa habilita la siguiente cuando supera su *quality gate*.

Paso	Comando	Qué hace	Qué obtenés
1	—	Instalación e inicio del proyecto	Proyecto configurado con los agentes
2	<code>/interview-me</code>	Entrevista sobre tu idea de investigación	Especificación de investigación de ocho secciones
3	<code>/discover</code>	Revisión de literatura y disponibilidad de datos	Revisión por ejes, bibliografía anotada y factibilidad de datos
4	<code>/strategize</code>	Estrategia de identificación	Strategy Memo y registro de decisión (gate: 80)
5	<code>/analyse</code>	Limpieza de datos y análisis empírico	Datos procesados, scripts, tablas y figuras
6	<code>/write</code>	Redacción del artículo en LaTeX	Secciones o paper completo
7	<code>/review</code>	Revisión por pares simulada	Reportes de los <i>critics</i>
8	<code>/talk</code>	Preparación de una presentación	Diapositivas
9	<code>/submit</code>	Adaptación al formato de una revista	Paquete de replicación y puntaje final (gate: 95)
10	<code>/dashboard,</code> <code>/checkpoint</code>	Trazabilidad y control (todo el proceso)	Diario, estado del proyecto y tablero

2. La rutina paso a paso

Cada paso tiene el mismo formato: qué hacés, qué escribís, qué obtenés y qué tenés que revisar vos. Esa última parte es la más importante.

Paso 1. Instalación e inicio

Qué hacés

1. Instalá la aplicación de escritorio de Claude y Git.
2. Hacé una copia de seguridad de la carpeta donde vas a trabajar e iniciá Claude Code ahí.
3. Pegá la instrucción de arranque (más abajo).
4. Cuando te lo pida, completá el nombre del proyecto y tu filiación institucional.

Qué escribís

```
Leé claude.md e instalá todos los agentes necesarios para hacer /discover, etc.
```

Qué obtenés

- El proyecto configurado, con todos los agentes instalados y listos para usar.

Qué tenés que revisar vos

- Que estés trabajando sobre una copia y no sobre tus archivos originales.

Paso 2. Prompt inicial y entrevista (/interview-me)

Qué hacés

1. Contale tu objetivo de investigación en lenguaje natural, como se lo contarías a un colega.
2. Activá el comando `/interview-me`.
3. Respondé la entrevista: el sistema estructura el diálogo hasta armar la especificación.

Qué escribís

```
Quiero investigar [tema]. Mi pregunta es [pregunta]. Trabajo con [países, período, datos que ya tengas].  
/interview-me
```

El primer bloque es un modelo para adaptar. En la ponencia el planteo fue el rol de los shocks externos en un conjunto de economías latinoamericanas.

Qué obtenés

- Una **especificación formal de investigación de ocho secciones**. Es la base de todo lo que viene después: a partir de ella se construyen el perfil de dominio, la revisión de literatura y el registro de decisiones.

Qué tenés que revisar vos

- Leé la especificación completa y corregí lo que no refleje tu idea antes de seguir. Todo el *pipeline* se apoya en este documento.

Paso 3. Revisión de literatura y datos (/discover)

Qué hacés

1. Corré `/discover` para obtener la revisión de literatura.

2. Corré la variante de datos, `/discover-data`, para evaluar qué series podés conseguir.

Qué escribís

```
/discover  
/discover-data
```

Qué obtenés

- Una **revisión de literatura completa**, organizada por ejes temáticos. Ordena los trabajos por importancia relativa para tu proyecto e indica título, revista, contribución, estrategia de identificación, hallazgos centrales y motivo de relevancia.
- Una **tabla resumen** y una **bibliografía anotada** que clasifica cada trabajo en siete categorías: relacionado directamente; mismo método pero contexto diferente; mismo contexto pero diferente método; fundamentos teóricos; papers de método; fuentes históricas de datos; y narrativas.
- Con `/discover-data`, un **análisis de factibilidad de datos**: para cada serie necesaria, la fuente, la cobertura y el grado de viabilidad.

Qué tenés que revisar vos

- **El puntaje de relevancia**. El sistema pondera más la afinidad técnica y econométrica que la explicación analítica o histórica, y penaliza en bloque a los trabajos más cualitativos o teóricos.
- **El diagnóstico de datos**. Tiende a sobreestimar lo que efectivamente existe y está disponible.
- Chequeá a mano que las referencias existan.

Paso 4. Estrategia de identificación (`/strategize`)

A partir de la especificación, el comando propone un diseño metodológico y, a la vez, actúa como crítico de su propia propuesta.

Qué hacés

1. Corré `/strategize`.
2. Esperá a que termine la revisión del crítico y leé los dos documentos que produce.

Qué escribís

```
/strategize
```

Qué obtenés

- **Strategy Memo**: el documento técnico que el agente programador va a implementar al pie de la letra. Incluye el estimando, la especificación, los supuestos, un plan de robustez de 22 ítems, las amenazas a la identificación, pseudo-código y las cinco objeciones de referato anticipadas. Es largo: está pensado para el siguiente agente (y para un referato exigente), no para una lectura rápida.
- **Registro de decisión**: un documento breve que deja constancia de por qué se eligió ese diseño y no otro.

Quality gate

El strategist-critic (con acceso de solo lectura al memo) puntúa el diseño en cuatro fases: diseño de identificación, validez, inferencia y pulido. Por debajo de 80, el sistema se niega a despachar la etapa siguiente.

Qué tenés que revisar vos

- **El gate se resuelve entre agentes.** No tenés que aprobar el memo para que el pipeline avance: si no lo leés y no disentís vos, nadie lo va a hacer.
- El puntaje certifica que no hay fallas de identificación no resueltas. No certifica que los resultados vayan a ser correctos ni que estés de acuerdo con el enfoque elegido.
- **Recomendación:** no partas (necesariamente) de la propuesta metodológica del sistema. La técnica que se elige no es neutral respecto de la pregunta de investigación. Usá el memo y el código como doble chequeo de una decisión que ya tomaste vos, y para enriquecer tu abanico de posibilidades técnicas.

Paso 5. Limpieza de datos y análisis empírico (/analyse)

Una vez superado el gate anterior, el orquestador despacha en paralelo a dos agentes, que comparten un mismo requisito: el memo de estrategia aprobado.

Qué hacés

1. Corré /analyse.
2. El **data-engineer** organiza primero los datos descargados en una carpeta de trabajo ("raw"). Cuando hay acuerdo con el coder sobre la estructura, los limpia y los ordena en el directorio de datos procesados.
3. En paralelo, el **coder** implementa lo que dice el memo (en el proyecto de la ponencia: el SVAR país por país, la descomposición de la varianza del error de pronóstico, la comparación entre regímenes y el plan de robustez de 22 ítems) y produce scripts, tablas y figuras.
4. El **coder-critic** revisa a ambos: calidad de código, reproducibilidad y, sobre todo, fidelidad al memo (que el código haga exactamente lo especificado y no una versión simplificada).

Qué escribís

| /analyse

Qué obtenés

- Datos limpios y ordenados, scripts, tablas y figuras.

Qué tenés que revisar vos

- **Podés intervenir a mano** en la carpeta de datos en cualquier momento, agregando archivos nuevos o corrigiendo los existentes.
- **Desacople silencioso.** Como los dos agentes trabajan en paralelo, uno puede asumir un nombre de columna o una unidad distinta de la que definió el otro. Revisá **todos** los archivos de salida de cada fase, no solo los que resumen el resultado final.

Paso 6. Escritura (/write)

La escritura se hace en LaTeX y se puede usar de tres maneras.

Qué hacés

1. Recomendado: arrancá extrayendo tu guía de estilo a partir de trabajos anteriores tuyos.
2. Elegí si querés una sección puntual o el paper completo.
3. Si usás /write full, leé y corregí en cada una de las tres paradas antes de seguir.

Qué escribís

```
/write style-guide  
/write [section] or  
/write full
```

Qué obtenés

- `/write [section]` redacta una sección puntual siguiendo una plantilla de «movimientos argumentativos» (qué tipo de párrafo va primero, cómo se narra un resultado, etc.).
- `/write full` redacta todo el paper en secuencia y se detiene en tres instancias: (1) introducción y posicionamiento en la literatura; (2) datos y estrategia empírica; (3) resultados, robustez y conclusión.
- `/write style-guide` extrae una guía de estilo de tus producciones académicas previas.
- En el proyecto de la ponencia salió un paper completo de 48 páginas (introducción, datos, estrategia empírica, resultados, robustez y conclusiones, más referencias), con 13 tablas y 12 figuras generadas a partir de lo que produjeron el data-engineer y el coder.

Qué tenés que revisar vos

- Que exista un paper completo **no significa que sea un buen paper**.
- Los sesgos de las etapas anteriores, sobre todo los de la estrategia de identificación, no quedan contenidos ahí: se trasladan a los resultados y a la escritura.

Paso 7. Revisión (/review)

El comando despacha al crítico que corresponde según el formato del archivo, y también admite revisiones puntuales que no dependen de esa detección automática.

Qué hacés

1. Corré `/review` sobre el archivo que quieras revisar, o agregá una opción para pedir una revisión puntual.
2. Leé los reportes que produce cada crítico.

Qué escribís

```
/review  
/review -methods  
/review -peer [revista]
```

Modos de revisión

Opción	Qué hace
<code>/review</code> sobre un <code>.tex</code>	Revisión comprehensiva con <code>strategist-critic</code> , <code>writer-critic</code> y <code>verifier</code> .
<code>/review</code> sobre un script	Dispara solo al <code>coder-critic</code> .
<code>-methods</code>	Audita la identificación causal en 4 fases.
<code>-proofread</code>	Pule el manuscrito.
<code>-code</code>	Revisa código aislado.
<code>-theory</code>	Audita pruebas formales.
<code>-replicate</code>	Compara una reimplementación en otro lenguaje.
<code>-all</code>	Corre todos los critics en paralelo, con un puntaje ponderado.
<code>-peer [revista]</code>	Simula un referato real (ver abajo).

El modo `-peer [revista]` es el más completo y simula un referato real en tres pasos: (1) el «editor» decide si rechaza de escritorio o manda a referato; (2) dos referees, ciegos entre sí, evalúan el paper; (3) el editor sintetiza todo en una decisión final: Accept, Minor, Major o Reject.

Qué tenés que revisar vos

- En todos los modos los critics **solo producen reportes: nunca editan nada**. Las correcciones las hacés vos o las pedís explícitamente.

Paso 8. Presentación (`/talk`)

Qué hacés

- Corré `/talk` y elegí el formato de salida (por ejemplo, Beamer).

Qué escribís

```
| /talk
```

Qué obtenés

- Una presentación. El agente **storyteller** arma las diapositivas y el **storyteller-critic** las revisa en cinco categorías: flujo narrativo, calidad visual, fidelidad de contenido, alcance para el formato y compilación.

Qué tenés que revisar vos

- Que cada afirmación de las diapositivas esté respaldada en el manuscrito.

Paso 9. Envío a publicación (`/submit`)

Qué hacés

- Corré `/submit`.
- Si el gate final marca problemas, corregí lo que indica y volvé a correrlo.

Qué escribís

```
| /submit
```

Qué obtenés

- Elige la **revista objetivo** a partir de tu perfil de investigador (que surge de la entrevista inicial).
- Arma el **paquete de replicación**: todo lo necesario para que otra persona pueda reproducir el paper.
- Un agente audita que la compilación funcione de principio a fin.
- **Gate final: 95 puntos**. Si no los supera, indica específicamente qué está fallando.

Qué tenés que revisar vos

- Aunque el gate se supere, la decisión de enviar el paper y la responsabilidad por lo que dice son tuyas.

Paso 10. Trazabilidad y control del proceso (/dashboard y /checkpoint)

Este paso no es una etapa más: es una capa de control que se actualiza a lo largo de todo el pipeline. El sistema lleva un **doble registro** y suma herramientas que podés usar en cualquier momento, sin haber completado una etapa específica.

Qué usás y para qué

Herramienta	Para qué sirve
<code>research_journal.md</code>	Diario de investigación, para vos . Registro narrativo de qué hizo cada agente y cuándo, con los puntajes y si se superó cada quality gate. Sirve para reconstruir las razones de cada decisión en una sesión futura, o al escribir la sección de metodología del paper, sin releer toda la conversación.
<code>pipeline_state.json</code>	Estado estructurado del proyecto, para el sistema . El orquestador lo consulta para saber en qué punto está el proyecto y qué agente despachar a continuación.
<code>/dashboard</code>	Vista unificada, tipo tablero, del estado del proyecto en formato HTML.
<code>/checkpoint</code>	Guarda el avance a memoria. Usalo antes de cerrar una sesión de trabajo.
Metaconsultas	Podés preguntarle al sistema por su propio estado para reorientarte sin memorizar la secuencia de comandos.
Git	Control de versiones: permite revisar el estado del trabajo en cualquier etapa anterior, no solo recuperar la configuración inicial.

Qué escribís

```
/dashboard  
/checkpoint
```

```
¿Cuáles serían los siguientes pasos usando los comandos de Clo-Author?
```

4. Buenas prácticas: uso responsable

Cook et al. (2026) proponen cuatro principios operativos para el uso de IA en investigación económica. Los formularon para el meta-análisis, y la ponencia los adopta para este tipo de flujos de trabajo:

- **Liderazgo humano**: dirigís activamente cada etapa, no la delegás por completo.

- **Responsabilidad humana:** la IA nunca puede ser coautora ni te exime de responder por lo producido.
- **Auditoría humana:** una porción significativa de las decisiones asistidas por IA se revisa y, cuando corresponde, se corrige.
- **Divulgación humana:** el uso de estas herramientas se declara explícitamente.

Documentá para poder replicar

- Registrá los **prompts** que usaste y la **versión exacta del modelo**.
- Cuando sea posible, exportá el registro de las consultas y las salidas generadas.
- Sin este registro, la naturaleza no determinística de los modelos y la obsolescencia acelerada de sus versiones hacen muy difícil reproducir la investigación con el paso del tiempo.

5. Glosario

Término	Qué es
Orquestador	Agente que recibe tus instrucciones, administra los flujos de trabajo y despacha al agente que corresponde en cada etapa.
Worker	Agente que produce un output (una revisión de literatura, un memo, un script).
Critic	Agente independiente que revisa el output de un worker. Solo produce reportes: nunca edita.
Quality gate	Umbral numérico que un output debe superar para habilitar la etapa siguiente.
strategist-critic	Puntúa el Strategy Memo en cuatro fases (gate: 80 puntos).
data-engineer	Organiza y limpia los datos: carpeta «raw» y directorio de datos procesados.
coder	Implementa el análisis empírico según el memo y produce scripts, tablas y figuras.
coder-critic	Revisa el trabajo del data-engineer y del coder: calidad de código, reproducibilidad y fidelidad al memo.
writer-critic	Participa en la revisión del manuscrito; en el modo <code>-proofread</code> , lo pule.
storyteller / storyteller-critic	Arman las diapositivas (<code>/talk</code>) y las revisan en cinco categorías.
Strategy Memo	Documento técnico con el diseño empírico, que implementa el coder
Registro de decisión	Documento breve que explica por qué se eligió un diseño y no otro.

6. Referencias

- Catelén, A. L. Automatizar sin perder el criterio: experiencias con IA en el proceso de investigación económica. Ponencia presentada en el CIGU.
- Cook, N., Bartoš, F., Bom, P. R. D., et al. (2026). Guidance for the use of AI in the meta-analysis of economics research. *Journal of Economic Surveys*. <https://doi.org/10.1111/joes.70105>
- Santanna, H. (s.f.). *clo-author: A Claude Code scaffold for empirical economics research* [Software]. GitHub. <https://github.com/hugosantanna/clo-author>